

WEEK-10SKILL

Task1:

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
UW PICO 5.09 File: signal_handler.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig, siginfo_t *info, void *context)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Ctrl+C detected\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received\n");
    }

    printf("Signal number: %d\n", sig);
    printf("Sender PID: %d\n", info->si_pid);
}

int main()
{
    struct sigaction sa;

    sa.sa_sigaction = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_SIGINFO;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("Signal handler program started\n");
    printf("PID: %d\n", getpid());

    printf("\n\nWaiting for signals...\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
Last login: Sun Sep 27 18:57:20 on ttys001
(eval):8: command not found: >
vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/week10skill
vishalkarnati@VISHALS-MacBook-Air week10skill % nano signal_handler.c
vishalkarnati@VISHALS-MacBook-Air week10skill % gcc signal_handler.c -o signal_handler
vishalkarnati@VISHALS-MacBook-Air week10skill % ./signal_handler
Signal handler program started
PID: 1326

Waiting for signals...
^C
SIGINT received - Ctrl+C detected
Signal number: 2
Sender PID: 1253
```

Task2:

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
UW PICO 5.09 File: shell_sigint.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void shell_sigint_handler(int sig)
{
    printf("\nShell received SIGINT, but will continue running.\n");
}

int main()
{
    char command[100];

    struct sigaction sa;

    sa.sa_handler = shell_sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    /*
     * Shell handles SIGINT instead of terminating.
     */
    sigaction(SIGINT, &sa, NULL);

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        if (strlen(command) == 0)
        {
            continue;
        }

        pid_t pid = fork();

        if (pid == -1)
        {
            perror("fork");
            continue;
        }

        if (pid == 0)
        {
            /*
             * Child gets the default SIGINT behavior.
            */
            exit(1);
        }
    }
}
```

```
UW PICO 5.09 File: shell_sigint.c Modified
fflush(stdout);

fgets(command, sizeof(command), stdin);
command[strcspn(command, "\n")] = '\0';

if (strcmp(command, "exit") == 0)
{
    printf("Shell exiting...\n");
    break;
}

if (strlen(command) == 0)
{
    continue;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    continue;
}

if (pid == 0)
{
    /*
     * Child gets the default SIGINT behavior.
     */
    signal(SIGINT, SIG_DFL);

    /*
     * Run the command.
     */
    char *args[10];

    int count = 0;
    char *token = strtok(command, " ");

    while (token != NULL && count < 9)
    {
        args[count] = token;
        count++;

        token = strtok(NULL, " ");
    }

    args[count] = NULL;
    execvp(args[0], args);

    perror("exec");
    exit(1);
}

/*
 * Parent waits for foreground child.
*/
waitpid(pid, NULL, 0);
return 0;
}
```

```
UW PIC0 5.09 File: shell_sigint.c Modified
signal(SIGINT, SIG_DFL);

/*
 * Run the command.
 */
char *args[10];

int count = 0;

char *token = strtok(command, " ");

while (token != NULL && count < 9)
{
    args[count] = token;
    count++;

    token = strtok(NULL, " ");
}

args[count] = NULL;

execvp(args[0], args);

perror("exec");
exit(1);
}

/*
 * Parent waits for foreground child.
 */
int status;

waitpid(pid, &status, 0);

if (WIFSIGNALED(status))
{
    printf("\nForeground process terminated by signal %d\n",
        WTERMSIG(status));
}
}

return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
Last login: Sun Sep 27 18:59:30 on ttys001
(eval):8: command not found: >
vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/os/week18skill
vishalkarnati@VISHALS-MacBook-Air week18skill % nano shell_sigint.c
vishalkarnati@VISHALS-MacBook-Air week18skill % gcc shell_sigint.c -o shell_sigint
vishalkarnati@VISHALS-MacBook-Air week18skill % ./shell_sigint
myshell> is
shell_sigint          shell_sigint.c          signal_handler          signal_handler.c          signal_handler.c'
myshell> sleep 30
^C
Shell received SIGINT, but will continue running.
myshell>
```

Task3:

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
UW PIC0 5.09 File: suspend_jobs.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 10

#define RUNNING 1
#define STOPPED 2
#define DONE 3

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    int state;
};

struct Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

/* Add a job to the job table */
void add_job(pid_t pid, char command[], int state)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    jobs[job_count].state = state;

    printf("[%d] PID %d %s %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           state == RUNNING ? "Running" : "Stopped",
           jobs[job_count].command);

    job_count++;
    next_job_id++;
}

/* Display all jobs */
void show_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == RUNNING)

```

⌘ Get Help	⌘ WriteOut	⌘ Read File	⌘ Prev Pg	⌘ Cut Text	⌘ Cur Pos
⌘ Exit	⌘ Justify	⌘ Where is	⌘ Next Pg	⌘ UnCut Text	⌘ To Spell

```
UW PICO 5.09 File: suspend_jobs.c Modified
if (job_count >= MAX_JOBS)
{
    printf("Job table is full\n");
    return;
}

jobs[job_count].job_id = next_job_id;
jobs[job_count].pid = pid;

strcpy(jobs[job_count].command, command);

jobs[job_count].state = state;

printf("[%d] PID %d %s\n",
        jobs[job_count].job_id,
        jobs[job_count].pid,
        state == RUNNING ? "Running" : "Stopped",
        jobs[job_count].command);

job_count++;
next_job_id++;
}

/* Display all jobs */
void show_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == RUNNING)
        {
            printf("[%d] PID %d Running %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);
        }
        else if (jobs[i].state == STOPPED)
        {
            printf("[%d] PID %d Stopped %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);
        }
        else
        {
            printf("[%d] PID %d Done %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);
        }
    }
}

/* Resume a stopped job */
void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
            Exit          Justify      Where is      Next Pg      UnCut Text    To Spell
        }
    }
}
```

```
UW PICO 5.09 File: suspend_jobs.c Modified
        printf("[%d] PID %d Running %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
    }
    else if (jobs[i].state == STOPPED)
    {
        printf("[%d] PID %d Stopped %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
    }
    else
    {
        printf("[%d] PID %d Done %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
    }
}

/* Resume a stopped job */
void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            if (jobs[i].state != STOPPED)
            {
                printf("Job is not stopped\n");
                return;
            }

            printf("Resuming job [%d]\n", job_id);

            if (kill(jobs[i].pid, SIGCONT) == -1)
            {
                perror("SIGCONT");
                return;
            }

            jobs[i].state = RUNNING;

            printf("[%d] PID %d Running %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);

            return;
        }
    }

    printf("Job not found\n");
}

int main()
{
    Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
    Exit          Justify      Where is      Next Pg      UnCut Text    To Spell
}
```

```
UW PICO 5.09 File: suspend_jobs.c Modified
{
    printf("Job is not stopped\n");
    return;
}

printf("Resuming job [%d]\n", job_id);
if (kill(jobs[i].pid, SIGCONT) == -1)
{
    perror("SIGCONT");
    return;
}

jobs[i].state = RUNNING;

printf("[%d] PID %d Running %s\n",
        jobs[i].job_id,
        jobs[i].pid,
        jobs[i].command);

return;
}

printf("Job not found\n");
}

int main()
{
    char command[100];

    /*
     * Shell should not be stopped by Ctrl+Z.
     * The child will use the default SIGTSTP behavior.
     */
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "jobs") == 0)
        {
            show_jobs();
        }
    }
}
```

```
UW PICO 5.09 File: suspend_jobs.c Modified

/*
 * Shell should not be stopped by Ctrl+Z.
 * The child will use the default SIGTSTP behavior.
 */
signal(SIGTSTP, SIG_IGN);
signal(SIGTTIN, SIG_IGN);
signal(SIGTTOU, SIG_IGN);

while (1)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        break;
    }

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        break;
    }

    if (strcmp(command, "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    if (strcmp(command, "resume ", 7) == 0)
    {
        int job_id;

        if (sscanf(command + 7, "%d", &job_id) == 1)
        {
            resume_job(job_id);
        }
        else
        {
            printf("Usage: resume <job_id>\n");
        }
    }

    continue;
}

/*
 * Only support:
 * start sleep
 */
if (strcmp(command, "start sleep") == 0)
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
    }
}
```

```
UW PICO 5.09 File: suspend_jobs.c Modified
}

if (strcmp(command, "resume ") == 0)
{
    int job_id;

    if (sscanf(command + 7, "%d", &job_id) == 1)
    {
        resume_job(job_id);
    }
    else
    {
        printf("Usage: resume <job_id>\n");
    }
}

continue;

/*
 * Only support:
 * start sleep
 */
if (strcmp(command, "start sleep") == 0)
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        /*
         * Put child in its own process group.
         */
        setpgid(0, 0);

        /*
         * Child should respond normally to Ctrl+Z.
         */
        signal(SIGTSTP, SIG_DFL);

        /*
         * Run sleep for 30 seconds.
         */
        execlp("sleep", "sleep", "30", NULL);

        perror("exec");
        exit(1);
    }

    /*
     * Parent also assigns child to its own process group.
     */
    setpgid(pid, pid);
}

/*
Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: suspend_jobs.c Modified

    tcsetpgrp(STDIN_FILENO, pid);

    int status;

    /*
     * WUNTRACED allows us to detect Ctrl+Z.
     */
    waitpid(pid, &status, WUNTRACED);

    /*
     * Shell gets terminal back.
     */
    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFSTOPPED(status))
    {
        printf("\nSIGTSTP detected\n");

        add_job(pid, "sleep 30", STOPPED);
    }
    else if (WIFEXITED(status))
    {
        printf("Process completed\n");
    }
}

continue;

printf("Unknown command\n");
printf("Use: start sleep | jobs | resume <job_id> | exit\n");
}

return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week10$ kill % nano suspend_jobs.c
vishalkarnati@VISHALS-MacBook-Air week10$ kill % gcc suspend_jobs.c -o suspend_jobs
vishalkarnati@VISHALS-MacBook-Air week10$ kill % ./suspend_jobs
myshe11> start sleep
sleep 30
^Z
SIGTSTP detected
[1] PID 1483 Stopped sleep 30
myshe11> jobs
[1] PID 1483 Stopped sleep 30
myshe11> resume 1
Resuming job [1]
[1] PID 1483 Running sleep 30
myshe11> exit
vishalkarnati@VISHALS-MacBook-Air week10$ kill %
```

Task4:

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
UW PICO 5.09 File: process_groups.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    /*
     * Shell ignores terminal control signals.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    printf("Shell PID : %d\n", getpid());
    printf("Shell PGID : %d\n", getpgid());

    printf("\nCreating child process...\n");

    pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Create a new process group.
         * Child becomes leader of its own group.
         */
        if (setpgid(0, 0) == -1)
        {
            perror("setpgid");
            exit(1);
        }

        printf("\n--- CHILD ---\n");

        printf("Child PID : %d\n", getpid());
        printf("Child PGID : %d\n", getpgid());

        printf("Child sleeping for 10 seconds...\n");

        sleep(10);

        printf("Child completed.\n");

        exit(0);
    }

    /*
     * Parent also makes sure the child

```

```
UW PICO 5.09 File: process_groups.c Modified
    if (pid == 0)
    {
        /*
         * Create a new process group.
         * Child becomes leader of its own group.
         */
        if (setpgid(0, 0) == -1)
        {
            perror("setpgid");
            exit(1);
        }

        printf("\n--- CHILD ---\n");

        printf("Child PID : %d\n", getpid());
        printf("Child PGID : %d\n", getpgid());

        printf("Child sleeping for 10 seconds...\n");

        sleep(10);

        printf("Child completed.\n");

        exit(0);
    }

    /*
     * Parent also makes sure the child
     * belongs to its own process group.
     */
    setpgid(pid, pid);

    printf("\n--- PARENT ---\n");

    printf("Child PID : %d\n", pid);

    printf("Giving terminal control to child group...\n");

    if (tcsetpgrp(STDIN_FILENO, pid) == -1)
    {
        perror("tcsetpgrp");
    }

    /*
     * Wait for child.
     */
    int status;

    waitpid(pid, &status, 0);

    /*
     * Restore terminal control to shell.
     */
    printf("\nChild finished.\n");

    printf("Restoring terminal control to shell...\n");

    if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1)

```

```
UW PICO 5.09 File: process_groups.c Modified

/*
 * setpgid(pid, pid);
 * printf("\n--- PARENT ---\n");
 * printf("Child PID : %d\n", pid);
 * printf("Giving terminal control to child group...\n");
 * if (tcsetpgrp(STDIN_FILENO, pid) == -1)
 * {
 *     perror("tcsetpgrp");
 * }
 *
 * /*
 * * Wait for child.
 * */
 * int status;
 *
 * waitpid(pid, &status, 0);
 *
 * /*
 * * Restore terminal control to shell.
 * */
 * printf("\nChild finished.\n");
 * printf("Restoring terminal control to shell...\n");
 * if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1)
 * {
 *     perror("tcsetpgrp");
 * }
 *
 * printf("Shell has terminal control again.\n");
 * return 0;
 */
}

Get Help      WriteOut      Read File    Prev Pg      Cut Text     Cur Pos
Exit          Justify        Where is     Next Pg      UnCut Text   To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week10$ kill % nano process_groups.c
vishalkarnati@VISHALS-MacBook-Air week10$ kill % gcc process_groups.c -o process_groups
vishalkarnati@VISHALS-MacBook-Air week10$ kill % ./process_groups
Shell PID : 1502
Shell PGID : 1502

Creating child process...

--- PARENT ---
Child PID : 1503
Giving terminal control to child group...

--- CHILD ---
Child PID : 1503
Child PGID : 1503
Child sleeping for 10 seconds...
Child completed.

Child finished.
Restoring terminal control to shell...
Shell has terminal control again.
vishalkarnati@VISHALS-MacBook-Air week10$ kill %
```